

GridPulse

Smart Grid Monitoring & AI Incident Dispatch System

Comprehensive System Architecture, Technical Documentation, Database Schema, AI Integration, and DevOps Deployment Guide.

System Version: 1.0.0 (Enterprise WAR)

Target Runtime: Apache Tomcat 10 | Java 17/21 | AWS RDS MySQL 8.4

AI Engine: Groq API (Llama-3.3-70b-versatile via LangChain4j)

Generated Date: July 2026

1. Executive Summary & Problem Statement

Modern electrical utility grids require continuous vigilance across wide geographical areas. In traditional grid management, substation faults such as transformer overheating, severe voltage drops, and overcurrent spikes are identified manually or through delayed alarms. This results in slow incident resolution times, risk of widespread blackouts, and uncoordinated technician dispatches.

GridPulse is an enterprise-grade, real-time smart grid monitoring and automated incident dispatch platform. It bridges grid telemetry with Artificial Intelligence to automatically detect thermal and electrical anomalies, diagnose probable root causes in real time, and dispatch specialized technicians based on workload and proximity.

Key Highlighting Innovations:

- **28 Indian State Grids Telemetry:** Real-time tracking of voltage, current, and temperature across major state substations.
- **Sub-Second LLM Fault Diagnosis:** Instant root cause classification powered by Groq Llama 3.3 70B AI via LangChain4j.
- **Automated Workload-Balanced Dispatch:** Automatic matching of repair tickets to technicians based on specialization, status, and active workload.
- **Unified Production Architecture:** Embedded React 18 SPA compiled inside a single enterprise Spring Boot 3 WAR artifact (`GridPulse.war`), deployed on Apache Tomcat 10 with AWS RDS MySQL.

2. Core System Architecture & Technology Stack

GridPulse uses a modern, decoupled layer architecture that is unified into a single WAR package for seamless enterprise cloud deployment.

Layer	Technologies / Frameworks	Key Responsibilities
Frontend UI	React 18, Vite, Vanilla CSS, Tailwind, Lucide React, Recharts	Single Page Application (SPA), interactive grid dashboards, telemetry visualization, role-specific UI views.
Backend API	Java 17/21, Spring Boot 3.3.1, Spring Security, Spring Data JPA	REST APIs, WebSocket live alerts, JWT authentication, business logic, transaction management.
AI Engine	LangChain4j 0.31.0, Groq API (Llama 3.3 70B Versatile)	Telemetry anomaly parsing, root cause analysis, priority assignment, repair recommendation generation.
Database	AWS RDS MySQL 8.4 Community Edition	Relational storage for substations, users, repair tickets, technician availability, and telemetry logs.
Deployment	Apache Tomcat 10, AWS EC2, GitHub Actions, Logback	Jakarta EE 10 deployment, daily rolling logs (gridpulse.log), automated CI/CD pipeline.

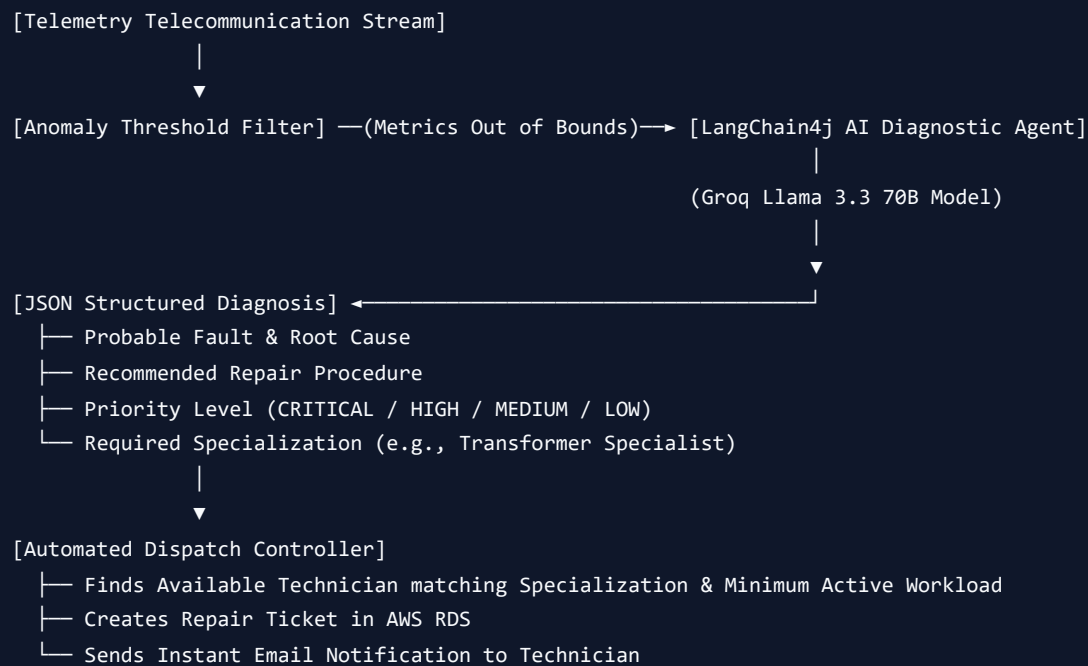
3. User Roles & Permission Matrix

GridPulse enforces Role-Based Access Control (RBAC) across three distinct operational roles:

- ADMIN System Administrator:** Full administrative privileges. Responsible for registering employees/technicians, managing user-account linkages, configuring grid thresholds, and viewing system audit logs.
- OPERATOR Grid Control Room Operator:** Monitors national grid telemetry, reviews AI-generated incident reports, manually overrides ticket assignments, and triggers emergency grid shutdowns.
- TECHNICIAN Field Service Technician:** Dedicated field portal view. Receives assigned repair tickets, updates repair status (‘ASSIGNED’ → ‘IN_PROGRESS’ → ‘RESOLVED’), and submits completion reports.

4. AI Automated Incident Dispatch Engine

When telemetry metrics cross defined thresholds (e.g., Temperature > 85°C, Voltage < 180V or > 250V, Current > 45A), the automated diagnostic pipeline triggers:



5. Database Schema & Data Model

The system utilizes 5 core relational tables in MySQL 8.4:

5.1 Table: `users`

Column	Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique user ID
username	VARCHAR(50)	UNIQUE, NOT NULL	Authentication login handle
password	VARCHAR(255)	NOT NULL	BCrypt hashed password
email	VARCHAR(100)	NOT NULL	User email address
role	VARCHAR(20)	NOT NULL	ADMIN, OPERATOR, or TECHNICIAN
full_name	VARCHAR(100)	NOT NULL	User display name

5.2 Table: substations

Column	Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Substation ID
name	VARCHAR(100)	NOT NULL	State Grid Substation Name
location	VARCHAR(100)	NOT NULL	City & State location
status	VARCHAR(20)	NOT NULL	NORMAL, WARNING, or CRITICAL
voltage	DOUBLE	NOT NULL	Live Voltage reading (V)
current	DOUBLE	NOT NULL	Live Current reading (A)
temperature	DOUBLE	NOT NULL	Live Temperature (°C)
max_capacity_kw	DOUBLE	NOT NULL	Maximum grid capacity in kW

5.3 Table: technicians

Column	Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Technician Profile ID
user_id	BIGINT	FOREIGN KEY (users.id)	Linked login user account
name	VARCHAR(100)	NOT NULL	Technician full name
specialization	VARCHAR(100)	NOT NULL	Domain (Transformer, Cable, High-Voltage)
status	VARCHAR(20)	NOT NULL	AVAILABLE, ON_DISPATCH, OFF_DUTY
active_tickets_count	INT	DEFAULT 0	Active workload tracking count

5.4 Table: `repair_tickets`

Column	Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Incident Ticket ID
substation_id	BIGINT	FOREIGN KEY (substations.id)	Target Substation ID
technician_id	BIGINT	FOREIGN KEY (technicians.id)	Assigned Technician ID
probable_fault	TEXT	NOT NULL	AI Root Cause Analysis Summary
recommended_repair	TEXT	NOT NULL	AI Recommended Repair Steps
priority	VARCHAR(20)	NOT NULL	CRITICAL, HIGH, MEDIUM, LOW
status	VARCHAR(20)	NOT NULL	ASSIGNED, IN_PROGRESS, RESOLVED
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Ticket creation timestamp

6. API Endpoints Reference

HTTP Method	Endpoint URL	Auth Required	Description
POST	/api/auth/login	No	Authenticates user and returns JWT token + role info.
POST	/api/auth/register	Yes (ADMIN)	Registers new employee/technician user account.
POST	/api/auth/forgot-password/send-otp	No	Generates and sends 6-digit OTP to user email.
POST	/api/auth/forgot-password/reset	No	Verifies OTP and updates user password.
GET	/api/substations	Yes	Returns live status for all 28 state grid substations.
GET	/api/substations/{id}	Yes	Returns detailed telemetry metrics for a specific substation.
GET	/api/tickets	Yes	Retrieves list of repair tickets (filtered by role).
PUT	/api/tickets/{id}/status	Yes (TECHNICIAN)	Updates ticket status ('IN_PROGRESS' or 'RESOLVED').
GET	/api/technicians	Yes (ADMIN/OPERATOR)	Retrieves active technician workload directory.
GET	/actuator/health	No	Production monitoring health check endpoint.

7. DevOps, CI/CD & Production Deployment

GridPulse is configured for single WAR deployment ('GridPulse.war') on Apache Tomcat 10 backed by AWS RDS MySQL.

7.1 Production Environment Configuration (`setenv.sh`)

```
# Location: /opt/tomcat10/bin/setenv.sh
export JAVA_OPTS="-Dspring.profiles.active=prod -Djava.awt.headless=true"

# AWS RDS Database Connection
export SPRING_DATASOURCE_URL="jdbc:mysql://gridpulse-db.cxy8c02umj4p.ap-south-1.rds.amazonaws.com:3306/gridpulse?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC"
export SPRING_DATASOURCE_USERNAME="root"
export SPRING_DATASOURCE_PASSWORD="Vamsi1122#"

# Security Secrets & Groq AI Key
export JWT_SECRET="9a67471a79b16bb185cfb52e399588b444747c32bf2542a1772f4e24cf8fef7f"
export GROQ_API_KEY="gsk_Gf6uJgLSV5ekwkg6Cw1xWgdyb3FYRZvoyqIC19L32pAROUNNipH"
export CORS_ALLOWED_ORIGINS="http://localhost:5173,http://<EC2-PUBLIC-IP>:8080"
```

7.2 Automated Build Scripts

Cross-platform scripts bundle the compiled React static assets into `src/main/resources/static` before executing Maven WAR packaging:

- `./build-war.ps1` (Windows PowerShell automation script)
- `./build-war.sh` (Linux/CI Bash script)

7.3 GitHub Actions CI/CD Pipeline (`.github/workflows/build.yml`)

```
Developer Push to Main → GitHub Actions Workflow Triggers
├─ Step 1: Checkout Code
├─ Step 2: Setup Java 21 & Node.js 20
├─ Step 3: `npm install` & `npm run build`
├─ Step 4: Copy React dist build to `backend/src/main/resources/static`
├─ Step 5: `mvn clean package`
├─ Step 6: SonarQube Code Quality Analysis Scan
└─ Step 7: Upload `GridPulse.war` Artifact
```